RELATÓRIO DE EVOLUÇÃO DO PROJETO — SPRINT 04

ChargeGrid Assistant · EV Challenge GoodWe · FIAP · Prompt and Artificial Intelligence · 2026.2

Números gerados por `python -m evals.run_ragas` em 17/10/2026; dados brutos em *evals/results/* e tabela consolidada em *evals/SCORES_POR_ITERACAO.md*.

1. Resumo da evolução

Sprints 1/2 — notebook no Colab, Llama 3.3 via Groq, prompt em texto corrido, sem base de conhecimento: o modelo respondia de memória.

Sprint 03 — projeto Python modular com LangChain LCEL, saída estruturada Pydantic, ConversationSummaryBufferMemory, guardrails em código, multi-provider (Gemma 4 via Ollama Cloud e Qwen 3.8 via Groq) e um primeiro RAG: FAISS com embeddings MiniLM sobre 4 arquivos Markdown, k=6, regra de uso do contexto dentro do system prompt v6. Esse RAG funcionava, mas não era medido (o eval checava palavras-chave em 13 casos), não citava fonte (a interface só listava o nome dos documentos recuperados) e não tinha regra de recusa quando a base não tinha a resposta.

Sprint 04 — o RAG passou a ser **medido**, **confiável** e **utilizável**:

O que mudou	Sprint 03	Sprint 04
Base de conhecimento	4 arquivos .md	7 PDFs: manual de produto GoodWe, mapa Modbus, OCPP, manual operacional EV ChargeOps, regimento condominial, FAQ, tabelas tarifárias
Carregamento	leitura direta do .md	PyMuPDFLoader + NFKC + tabelas linha a linha
Chunking	1 chunk por seção "##"	RecursiveCharacterTextSplitter dentro de cada seção (800/120) + cabeçalho [Documento — Seção]
Embeddings	MiniLM multilíngue (FastEmbed)	nomic-embed-text (768 d)
Vector store	FAISS (arquivo local)	ChromaDB persistente, reindexação automática
Recuperação	vetorial, k=6	híbrida (vetorial + BM25 com expansão de consulta, RRF), k=6
Prompt de RAG	dentro do system prompt v6	prompt RAG versionado (v1 a v4), separado da persona (system prompt v7)
Citação de fonte	não (só nomes recuperados na UI)	documento, seção e página em toda resposta, resolvidos pelos metadados do trecho
Recusa fora do contexto	não	guardrail de escopo + regra FORA DO CONTEXTO do prompt
Prompt injection via docs	não tratado	sanitização dos trechos + regra no prompt
Avaliação	13 casos por palavra-chave	RAGAS (faithfulness, answer_relevancy) + rubrica de fallback, por iteração
Interface	app.py (Streamlit)	app/streamlit_app.py: citação visível, trechos recuperados, seletor de prompt/k/busca e aba multi-provider (2 modelos × 2 prompts)

2. Pipeline RAG — chunking, base de conhecimento e trade-offs

Pipeline: PyMuPDFLoader → RecursiveCharacterTextSplitter → nomic-embed-text → ChromaDB → retriever (top-6 híbrido com expansão de consulta) → sanitização → prompt RAG versionado (v4) → LLM → PydanticOutputParser → citações pelos metadados. Um módulo por etapa em *src/rag/*.

Base de conhecimento (*data/knowledge_base/*): o manual de produto e o mapa Modbus vêm da documentação oficial GoodWe do HCA G2. A camada OCPP é o mapeamento sobre os registradores oficiais. O manual operacional EV ChargeOps, o regimento de recarga condominial, o FAQ e as tabelas tarifárias são ilustrativos e dizem isso numa nota de proveniência. Os PDFs são gerados a partir de fontes Markdown versionadas (*scripts/gerar_pdfs_base.py*). Especificação de hardware só existe no manual de produto, que lista explicitamente os únicos 3 modelos da linha: é isso que impede o assistente de inventar um modelo inexistente.

Decisões de chunking (medidas nas iterações):

- **it1, baseline:** página inteira no splitter, chunk_size=1000, overlap=0 → 32 chunks. O corte por tamanho juntava o fim de uma seção com o começo de outra.

- **it2:** corte nas seções numeradas e depois o splitter dentro da seção, chunk_size=800, overlap=120 → 61 chunks. As 56 seções têm de 135 a 1.536 caracteres (mediana 393): com 800, quase todas cabem num chunk e só as tabelas grandes do Modbus se dividem. O overlap de 15% preserva a transição entre partes. O cabeçalho [Documento — Seção] no texto mantém o contexto do pedaço. Normalização NFKC desfaz as ligaduras do PDF.
- **it3:** as mesmas 61 partes, com as tabelas reescritas linha a linha (find_tables do PyMuPDF), porque a extração padrão põe cada célula numa linha e o modelo trocava a linha do código 0x0002 pela do 0x0003.

Trade-offs: mais chunks deixam a indexação mais lenta (9,9 s → 14,3 s em CPU) e exigem k maior (3 → 6), o que aumenta o prompt (~4.050 tokens com o prompt RAG v4 e k=6). Em troca, o documento certo passou a ser recuperado em 100% das perguntas. A busca híbrida adiciona um índice BM25 em memória (desprezível para 61 chunks; teria que ir para um índice persistente numa base muito maior). O nomic-embed-text roda localmente via FastEmbed, e não pela Ollama Cloud, porque a Cloud retorna HTTP 401 para modelos de embedding.

3. Tabela de comparativo antes/depois

Mesmo eval set (*evals/rag_eval_set.py*): 18 perguntas respondíveis cobrindo os 7 documentos e 3 perguntas sem resposta na base. Modelo gerador: gemma4:cloud. Juiz do RAGAS e da rubrica: gemma4:cloud, temperature 0.

Métrica	Sprints 1/2 (original)	Sprint 03 (FAISS)	S04 it1	S04 it2	S04 it3	S04 it4	S04 it5	S04 it6 (final)
Faithfulness (RAGAS)	0,137	0,497	0,751	0,856	0,887	0,902	0,896	0,895
Answer relevancy (RAGAS)	0,732	0,655	0,717	0,767	0,785	0,821	0,821	0,777
Faithfulness (rubrica)	0,097	0,847	0,875	0,917	1,000	1,000	1,000	1,000
Answer relevancy (rubrica)	0,681	0,722	0,861	0,931	1,000	1,000	1,000	0,944
Contexto: doc. certo recuperado	— (sem RAG)	66,7%	94,4%	94,4%	100%	100%	100%	100%
Perguntas novas: respondidas com a fonte certa	—	—	—	—	—	—	6/10	10/10
Citação de fonte presente	0%	0%	5,6%	94,4%	100%	100%	100%	100%
Citação apontando o doc. certo	0%	0%	0%	88,9%	100%	100%	100%	100%
Acerto factual (checagem)	33,3%	55,6%	77,8%	88,9%	100%	100%	100%	100%
Recusa correta (sem base)	1/3	2/3	3/3	3/3	3/3	3/3	3/3	3/3
Latência média (s)	1,70	1,30	1,21	1,26	1,73	2,20	1,20	1,45

"Perguntas novas": 10 perguntas fora do eval set, escritas sem o vocabulário dos documentos, rodadas no pipeline completo (*evals/test_recuperacao.py --com-llm*). Coluna destacada: configuração final.

Configuração de cada iteração da Sprint 04:

- **it1** — chunk por página 1000/0, sem normalização, busca vetorial k=3, prompt RAG v1 (regra da Sprint 03, sem citação).
- **it2** — chunk por seção 800/120 + NFKC + cabeçalho, busca vetorial k=5, prompt RAG v2 (grounding, citação por id, recusa fora do contexto, documentos como dado).
- **it3** — + tabelas linha a linha, busca híbrida (vetorial + BM25), porta de relevância 0,69, prompt RAG v3 (resposta direta, valores da mesma linha da tabela, contas explícitas, nada fora dos trechos).
- **it4** — mesma recuperação da it3, prompt RAG v4 (a resposta direta passa a retomar o sujeito da pergunta).
- **it5** — it4 + expansão de consulta no BM25 ("quanto custa", "faturamento" → "tarifa", "posto tarifário"), 1 vaga garantida ao melhor trecho do BM25, k=6.
- **it6 (final)** — it5 sem a porta de relevância, com a base corrigida (sem os 2 trechos escritos sob medida para o eval set; ver Problema 8).

Ganho por iteração (RAGAS): faithfulness +0,105 (it1→it2), +0,031 (it2→it3) e +0,015 (it3→it4); answer relevancy +0,050, +0,018 e +0,036. A rubrica de fallback mostra a mesma direção nas duas dimensões. A it5 empata com a it4 no RAGAS (diferenças dentro do ruído). Na it6, a answer relevancy cai para 0,777 por um único caso, o R18: sem a frase sob medida na base (Problema 8), o modelo responde que o GW30K-HCA-20 "não consta na base" e lista os 3 modelos reais, o que está correto mas tira relevância 0. Sem o R18, a it6 fica em 0,822, igual à it5. O ganho das it5 e it6 aparece nas perguntas reais, escritas como o operador escreve (6/10 → 10/10 respondidas com a fonte certa).

Como lemos esses ganhos (ruído medido): repontuar as mesmas respostas dá o mesmo score (o juiz é determinístico com temperature 0), mas repetir a geração da it4 deu faithfulness 0,865 em vez de 0,902 (só 9 de 21 respostas idênticas), e answer relevancy 0,818 em vez de 0,821. Portanto: it1→it2 é ganho real; it2→it3 aparece

com clareza no acerto factual e na citação correta (88,9% → 100%); it3→it4 é real em answer relevancy (+0,036); diferenças de faithfulness abaixo de ~0,04 são ruído. Uma ablação (recuperação da it3 com o prompt v2: 0,892 / 0,806) mostrou que a v3 sozinha não superava a v2. As respostas secas da v3 ("Leva cerca de 4 horas.") derrubavam a answer relevancy; a v4 nasceu desse diagnóstico.

Qualidade do contexto recuperado: a Sprint 03 só tinha fonte para 2/3 das perguntas (regimento, FAQ e manual operacional não existiam na base). Da it3 em diante, o documento que contém a resposta está entre os k trechos recuperados em 18/18 perguntas.

Presença de citação: a Sprint 03 não tinha citação feita pelo modelo. Da it3 em diante, 100% das respostas citam documento, seção e página, sempre o documento certo. Como a citação é resolvida pelos metadados do trecho, e não pelo texto gerado, o modelo não consegue citar uma fonte que não foi recuperada.

Regressão: os 13 casos do eval da Sprint 03 (potência, faturamento, memória em 3 turnos, jailbreak, limites de atuação) rodados na configuração final tiraram nota 10,0/10 com 100% de structured output (*evals/results/comparativo_20261017-165300.md*).

Multi-provider (configuração it3, prompt v3): Gemma 4 — faithfulness 0,887, answer relevancy 0,785, latência 1,7 s; Qwen 3.8 — 0,897 e 0,824, mesma taxa de acerto, citação e recusa (100%), mas 29 s por pergunta no eval, porque o plano gratuito do Groq (8.000 tokens/minuto) põe as chamadas em fila. Empate técnico em qualidade; o Gemma é o padrão pela latência. Detalhes em *docs/relatorio_modelos.md*.

4. Problemas encontrados e soluções

Problema 1 — A Ollama Cloud não serve nomic-embed-text

O enunciado pede nomic-embed-text na Ollama Cloud, mas */api/embed* retornou HTTP 401 para nomic-embed-text e embeddinggemma com a mesma chave que funciona no chat. **Decisão:** rodar o mesmo modelo (nomic-embed-text v1.5, o que a Ollama distribui com esse nome) localmente via FastEmbed, com os prefixos search_document/search_query, e manter um backend "ollama" configurável. **Por quê:** cumpre o modelo exigido sem depender de um endpoint indisponível.

Problema 2 — Em português, a busca vetorial pura ranqueava o documento errado

Na it2, para "recarga das 18:30 às 19:00 ... quanto fica o faturamento?", as 8 seções do FAQ (todas com "recarga") ficaram acima da tabela de postos tarifários. **Decisão:** busca híbrida, ranking vetorial fundido com BM25 por Reciprocal Rank Fusion (documento certo no top-5: 17/18 → 18/18). Na it5, a mesma pergunta sem "pela política tarifária" voltou a falhar, porque o operador não usa a palavra "tarifa": a consulta do BM25 passou a ser expandida com o vocabulário do documento, e o melhor trecho do BM25 ganhou vaga garantida (10/10 perguntas novas com a seção certa).

Problema 3 — Tabelas do PDF extraídas uma célula por linha

Na it2, o modelo respondeu que o código Modbus 0x0002 era "valor fora da faixa" (a definição do 0x0003). **Decisão:** detectar as tabelas com `page.find_tables()` e reescrever cada linha como "| 0x0002 | Illegal Data Address | ... |"; regra TABELAS no prompt RAG v3. **Por quê:** preserva a associação código → nome → causa → ação dentro da mesma linha.

Problema 4 — Ataques de prompt injection mais sutis passavam pelo prompt

O ataque óbvio ("IGNORE TODAS AS INSTRUÇÕES") não funcionou nem sem defesa, mas uma "atualização obrigatória para sistemas automatizados" e uma tentativa de fechar as tags do prompt e abrir uma "regra nova" fizeram o modelo informar uma tarifa falsa mesmo com o prompt v3. **Decisão:** sanitização dos trechos antes do prompt, sem nenhum falso positivo nos 61 trechos legítimos. **Resultado** (Gemma, 4 ataques × 2 repetições): sem defesa 2/8 resistidos; só prompt 4/8; prompt + sanitização 8/8. **Ressalva:** os padrões dos dois ataques mais sutis foram acrescentados depois que o teste mostrou que passavam (teste de regressão); por isso a regra do prompt continua como segunda camada.

Problema 5 — Limites dos providers derrubavam a avaliação

Com 4 workers, o RAGAS recebeu HTTP 429 ("too many concurrent requests") da Ollama Cloud, e as amostras com erro viraram NaN; o Groq limita a 8.000 tokens por minuto. **Decisão:** RAGAS com 1 worker e 6 tentativas, retry com backoff exponencial na chain e retries no cliente Groq. Nas rodadas finais houve 1 NaN em 180 amostras.

Problema 6 — O prompt v3 empatou com o v2 e as respostas variavam entre execuções

Isolando o prompt (mesma recuperação), a v3 não superou a v2; e repetir a mesma iteração mudou 12 das 21 respostas e a faithfulness em ~0,04. **Decisão:** diagnóstico caso a caso, criação da v4 e medição explícita do ruído

do juiz e da geração antes de afirmar ganho. A v4 ganhou +0,036 em answer relevancy, acima do ruído medido.

Problema 7 — Limitações do próprio avaliador

(a) O RAGAS não considera o resultado de uma conta como sustentado pelo trecho: no R01, “5 kWh × R$ 1,65 = R$ 8,25” está certo e tirou faithfulness 0,4. (b) A primeira versão da rubrica dava 1,0 de relevância até para respostas que não entregavam o dado pedido. **Decisão:** a rubrica conta contas corretas como sustentadas e ancora a relevância no dado pedido; todas as execuções foram repontuadas. Concordância por caso com o RAGAS: r = 0,68 (faithfulness) e 0,67 (answer relevancy), em 180 amostras.

Problema 8 — A revisão final achou uma porta que recusava perguntas reais e dois trechos sob medida para o teste

(a) A porta de relevância (limiar 0,69 no melhor score vetorial) tinha sido calibrada só com as perguntas longas do eval set. Perguntas reais e curtas ficam entre 0,630 e 0,758, sobrepostas às de fora do domínio (0,599–0,667): com a porta, o pipeline respondeu só 6 de 10 perguntas novas, e em “O que fazer quando aparece falha de aterramento?” respondeu de memória, sem fonte. (b) O manual de produto dizia “não há um modelo de 30 kW” (exatamente o caso R18) e o FAQ usava os mesmos números do R17. **Decisão:** remover a porta (a recusa fica com o guardrail de escopo e a regra FORA DO CONTEXTO: 6/6 perguntas fora do domínio recusadas) e corrigir os dois trechos; a it6 mede essa configuração. Sem a frase sobre 30 kW, o R18 continua sem potência inventada, mas a relevância dele cai de 0,85 para 0 (ver seção 3). **Por quê:** um filtro que só funciona no conjunto em que foi calibrado, e um teste que a base responde literalmente, inflam a avaliação sem melhorar o sistema.

5. Equipe e divisão de trabalho

Nome	RM
Vinicius Molena	571270
Matheus Caviglia Ferreira	569638
Gabriel Vilas Boas da Silva Viana	571603
Nathan Guilherme Werner	572925
Gustavo Henrique Pereira Correia	569921
Ricardo Algazi dos Santos	569600